Decentralized Multispectral UAV Swarms for Search and Rescue: Early-Fusion Perception, Delay-Tolerant Networking, and Energy-Aware Self-Organization

Devesh Agarwal, Naman Chaudhary, Sankalp Mittal
Department of Information Technology
National Institute of Technology

Abstract—Searching a large disaster area quickly requires many aircraft that can see people in poor light, share what they find over unreliable radio links, and keep working when a member is lost. We present a simulated multi-UAV search-and-rescue system with three coupled parts: (i) a 4-channel RGB+thermal early-fusion YOLO11 person detector, (ii) a delay-tolerant ad-hoc mesh with a log-distance/shadowing radio model and store-carry-forward relaying of victim alerts, and (iii) a fully decentralized control layer in which every UAV plans from its own, possibly stale, view: energy-weighted centroidal Voronoi partitioning (EW-CVT) gated by dual-threshold hysteresis, sector claiming, and lease-based failure suspicion with no central planner. Each component has an on/off switch, and we measure what removing it costs over 1,320 paired missions. Buffering alerts raises the share of victims known to the operator from 3.4% to 98% at 5 km; forwarding shortens alert latency and protects alerts when the finder is lost; failover recovers 11 points of coverage after a UAV loss. Decentralization has a measurable price: after a loss, failover takes about 230 s and the mission about 133 s longer than with an omniscient planner. All results are from a kinematic simulation; we state the limits of that evidence.

Index Terms—UAV swarm, search and rescue, multispectral fusion, delay-tolerant networking, decentralized coverage control, centroidal Voronoi tessellation

I. Introduction

The first hours after an earthquake, flood or building collapse largely determine who is found alive. A swarm of small UAVs can cover a wide area much faster than ground teams, but three practical problems stand in the way. Perception: visible-light cameras fail in shadow, smoke and darkness, where people are still visible to a thermal camera. Communication: over kilometre-scale areas the swarm is mostly out of radio contact, so a design that assumes a connected network delivers almost nothing. Control: a single ground planner is a single point of failure and needs links that disaster areas rarely offer, yet each UAV acting on its own risks duplicated or abandoned work, especially when batteries differ and a UAV can be lost silently.

This paper studies these three problems together rather than separately, because they interact: what a UAV can decide depends on what it has actually heard, and what it hears depends on the radio and on where the others are. Our contributions are:

1) A 4-channel RGB+thermal early-fusion detector (YOLO11) and its evaluation under day, night and degraded-thermal conditions.
2) A delay-tolerant mesh in which victim alerts are buffered per recipient and carried onward by any UAV that receives them (store-carry-forward), evaluated against a physical radio model.
3) A decentralized control layer with no central planner: EW-CVT partitioning with dual-threshold hysteresis, claim-and-yield sector assignment, and lease-based failure suspicion that works when silence is the normal state.
4) A component-wise ablation over 1,320 paired simulated missions that reports effects with confidence intervals, including unfavourable ones, and states what the simulation cannot show.

II. Related Work

UAVs for search and rescue. Small UAVs have been used for wilderness and disaster search since the field trials of Goodrich et al. [19]; recent surveys cover sensing, task assignment, and path planning [18], and the single-UAV coverage problem specifically [20]. Multi-robot exploration and task allocation [21], [22] and auction-based methods such as CBBA [23] address who searches what. Those methods assume reliable communication or converge by consensus [24]; our setting is closer to the sparse, intermittent links of flying ad hoc networks [25], [26].

Multispectral person detection. Pairing visible and thermal imagery improves pedestrian detection in poor light [1]–[3], [28]. Early fusion, stacking the modalities as input channels, is the lightest option for edge hardware and is the one used here, on top of the YOLO family [4], [5], [29]. Speth et al. [6] showed RGB+thermal early fusion for person detection from a drone and released the multispectral dataset we use; our detector follows their early-fusion design.

Coverage control. Dividing an area among agents by Voronoi cells and moving each agent to its cell's centroid (Lloyd iteration) is a standard coverage method [7]–[9], and variants that need only limited-range interaction between agents exist [10], as do adaptive, fully decentralized laws [30]. Ours differs in that agents act on stale views

received over a lossy link and resolve conflicts by claiming. Weighting the distance by a per-agent quantity gives a power diagram [11]; we use the weight to encode battery. Hysteresis on such updates, to avoid chattering from noisy inputs, is our addition to the control loop.

Failure detection. Deciding that a peer is lost without a reliable channel is the unreliable failure detector problem [33]. Accrual detectors output a suspicion level from heartbeat arrival times [34]; that fails here because silence is normal in a disconnected mesh. Our sortie lease instead follows the lease idea of Gray and Cheriton [35]: a claim is valid until a stated time, after which it may be released.

Delay-tolerant networking. When end-to-end paths rarely exist, nodes store messages and forward them on contact [12]. Epidemic and spray-and-wait routing [13], [14] trade delivery against copies and buffer use, and probabilistic variants use contact history [31]; gossip-style dissemination has the same flavor [32]. We use epidemic forwarding with a buffer cap. UAV network simulators couple flight and network models [16]; our radio model is a simple log-distance channel [17]; air-to-ground links can be better than this when line of sight is likely [27], so ours is conservative. Our per-node broker follows the topic-subscription idea of distributed MQTT brokers with data-link look-up (DMLT) [15]; in the mission simulator all alerts share one topic, so only the broker's per-recipient buffering is exercised and topic filtering is not.

III. System Overview

Fig. 1 shows the system. A team of N UAVs launches from a common base and searches an $L \times L$ arena divided into sectors. Every UAV runs the same three components:

Perception turns a multispectral frame of the sector below into victim detections. Mesh communication sends alerts and periodic state beacons over a lossy radio link, buffers what could not be sent, and carries received alerts onward. Control chooses which sectors to search, from the UAV's own battery and its current view of its peers, and builds a route. The ground control station (GCS) is a passive sink: it receives alerts and plans nothing.

The interfaces are narrow: detections go to the mesh as alerts, beacons from the mesh update the control layer's view of the peers, and control hands a route to flight. We evaluate in a kinematic simulator with real frames from a multispectral aerial dataset in each sector; the detection, networking and control logic are the actual code under test, while the aircraft and radio are models.

IV. Method

A. Early-fusion perception

Visible RGB and thermal long-wave infrared frames are stacked into a four-plane tensor, and the first convolution of the detector takes four input channels. The channel order and input size follow the model family (the shipped model is YOLO11s at 1280 px; frames are 1280×937). The thermal kernel is initialised from the pretrained red-channel kernel, and training uses no colour jitter, which is meaningless for a thermal plane. A detection's box centre is mapped linearly into the bounds of the sector it was taken in to give a world coordinate; this is sector-based georeferencing, not a camera model.

B. Radio channel and delay-tolerant mesh

Channel. Path loss between two UAVs at distance d is

$$PL(d) = PL(d_0) + 10\,n \log_{10} \tfrac{d}{d_0} + X_\sigma + \Delta_{\text{obs}}, \quad (1)$$

where $\Delta_{\text{obs}} = 15\,\text{dB}$ when the link is obstructed (a random 10% chance beyond 50 m) and 0 otherwise, with $PL(d_0) = 20 \log_{10} f_{\text{MHz}} - 27.55$, $n = 2.5$, $d_0 = 1\,\text{m}$, log-normal shadowing X_σ with $\sigma = 4\,\text{dB}$, and a noise floor of $-90\,\text{dBm}$. SNR $= P_{tx} - PL - N_0$ maps to a packet error rate: PER $= 0$ above 25 dB, 1 below 5 dB, and $1 - (1 + e^{-0.5(\text{SNR}-15)})^{-1}$ between. Arenas of 2 km or more use 33 dBm at 2.4 GHz, giving PER ≈ 0.5 at about 520 m; smaller arenas use 30 dBm at 5 GHz (about 220 m). Against arenas of several kilometres, the swarm is mostly disconnected in flight.

Alerts. A UAV that detects a victim makes one direct attempt to every other UAV and to the GCS. Each failed copy is stored in a per-recipient QoS-1 buffer and flushed when the link to that recipient has PER < 0.2, in practice when the UAV returns to base.

Store-carry-forward. A UAV that receives an alert also keeps a copy for every recipient not known to hold it and hands it over whenever it later meets that recipient with a good link, so no UAV has to be in the right place at the right moment, and an alert survives the loss of its finder. On contact only copies the other side lacks are sent. A per-UAV buffer cap drops the oldest copy first.

Beacons. Every 5 s each UAV broadcasts its position, committed EW-CVT generator and weight, state, the time its route should bring it home, its sector claims and the set of sectors it knows are scanned. Beacons are single-hop and unbuffered: stale state is not worth delivering late. A receiver merges them; scanned sets are merged as grow-only sets, so coverage knowledge spreads transitively on contact.

C. Decentralized energy-aware control

EW-CVT partitioning. With battery B_i and capacity $B_{\max}$, UAV i has weight $w_i = (B_i/B_{\max})^\alpha$. A sector with centre c_s is preferred by the UAV with the smallest power distance

$$d(s,i) = \frac{\|c_s - g_i\|^2}{\max(0.01,\, w_i)}, \quad (2)$$

where g_i is the UAV's committed generator. $\alpha = 0$ is plain Voronoi, and larger α shrinks low-battery territories. The launch partition is the converged CVT (15 Lloyd iterations) for the launch batteries.

Dual-threshold hysteresis. Every 5 s each UAV computes the centroid C_i^* its cell should have, over the peers it

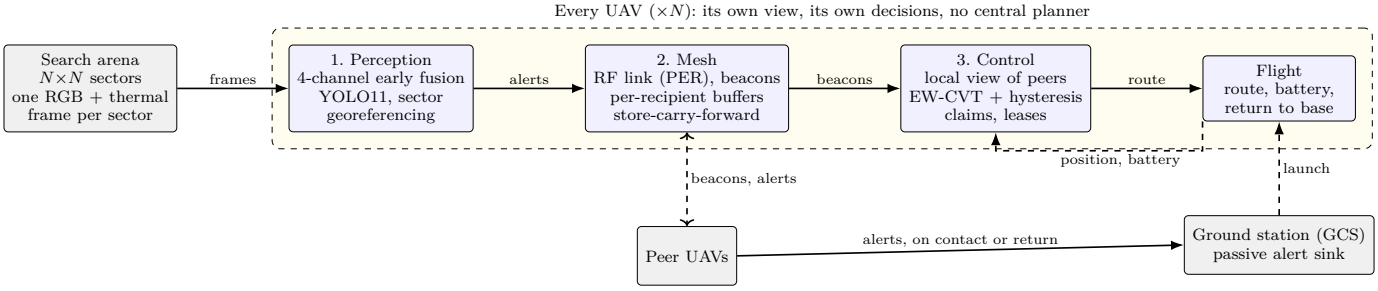


Fig. 1. System architecture. Each UAV carries the same three components and plans only from the beacons it has actually received; the ground station only receives alerts. Delivery to the ground station is mostly on return to base, or when a peer that holds a copy returns.

believes are active, using their last advertised generators and weights. It commits the new generator and weight only if

$$U_i = \big(\Delta B_i \geq \tau_\epsilon \ \wedge \ \|C_i^* - C_i\| \geq \tau_\delta\big) \ \vee \ \big(\mathcal{N}_i(t) \neq \mathcal{N}_i(t_k)\big), \tag{3}$$

with $\tau_\epsilon = 2\%$ of $B_{\max}$, $\tau_\delta = 5\,\text{m}$, and $\mathcal{N}_i$ the UAV's own view of the live set. Battery readings carry sensor noise; without the gate every tick would commit and borders would jitter.

Claims. A UAV claims a sector if it is the preferred owner by $d(s,i)$ using committed values, and nobody holds a valid claim on it. It keeps its existing claims unless a better claim (smaller d, ties by id) reaches it, and it may take a peer's claim only if it would arrive well before that peer's advertised arrival time. A UAV with nothing left claims the nearest unclaimed sector. Routes are greedy nearest-neighbour tours ending at the launch pad.

Failure suspicion by lease. In flight, silence says nothing about a peer: it may be out of range or have landed. A peer instead advertises when its route should bring it home; a peer still missing slack seconds after that (default 60 s) is suspected lost, and its claims are released. A UAV in flight cannot distinguish a lost peer from one that landed quietly, so released work is taken only by UAVs at base, which have heard the landed peers. Landed UAVs listen for 10 s, split the work by $d(s,i)$ among themselves, and relaunch if battery allows. A suspicion is withdrawn if the peer is heard again. A UAV at 15% battery aborts and returns; at 0% it is lost.

Oracle baseline. For comparison we keep a central planner that reads every UAV's true state, assigns all sectors by $d(s,i)$ every 5 s, and learns of a failure instantly. It bounds what decentralization could achieve.

Algorithm 1 Per-UAV control tick (every 5 s, own view only)

1: merge any beacons received since the last tick into the view (peers, claims, scanned set)
2: for each peer j overdue at base by more than slack: mark j suspected; release its claims
3: $\mathcal{N}_i \leftarrow$ self $\cup$ peers believed active (landed peers included only if self is at base)
4: read own battery B_i (noisy); $w_i^* \leftarrow (B_i/B_{\max})^\alpha$
5: $C_i^* \leftarrow$ EW-CVT centroid of my cell over $\mathcal{N}_i$ using peers' advertised (g_j, w_j)
6: if U_i (Eq. 3) then commit $g_i \leftarrow C_i^*$, $w_i \leftarrow w_i^*$
7: keep claims not beaten by a better claim; add free sectors I prefer by $d(s,i)$
8: if no claims left: claim nearest unclaimed sector; if at base: also share released work
9: if claims changed: rebuild nearest-neighbour route ending at the pad
10: advertise route ETA, claims and scanned set in the next beacon

V. Experimental Setup

Missions. Six UAVs search a 5×5 km arena of 25 sectors. Each sector shows one frame of the NII-CU multispectral aerial person-detection dataset (NII-CU) [6], with its labels as ground truth (36–56 victims per world). UAVs launch from a pad, fly at up to 50 m/s in transit and 32 m/s while searching, and drain 0.07%/s above 40 m/s and 0.04%/s otherwise. Launch batteries are uniform in 20–100%, and battery readings have noise $\sigma = 1.5$ points. In the loss scenario UAV 2 fails silently at $t = 80$ s; the others are not told.

Seeds and statistics. A seed fixes the frame in each sector and every launch battery, so each variant sees the same 20 worlds. We report paired differences with 95% confidence intervals; if an interval contains zero we do not claim an effect. Missions run to completion or 1,500 s. Detection inside missions is cached per frame, since each sector shows one static image.

Metrics. Coverage is the share of sectors scanned at least once. Precision/recall match reported victims to ground truth within 15 m (one-to-one), with recall taken over all victims in the arena. Operator recall counts only victims whose alert reached the GCS; alert latency is the delay from detection to GCS arrival. Failover latency is the time from the injected failure to the first survivor claiming one of its sectors. Duplicate scans count sectors scanned more than once. Sector flips count claims a UAV gave up to a better claim; route changes count route rebuilds; commits count hysteresis-approved updates.

Detector evaluation. We run the shipped model on 150 held-out validation frames (IoU 0.5, confidence 0.25). Night darkens RGB to 12% brightness and adds noise; thermal washout scales thermal contrast to 25%; RGB-only and thermal-only set the other plane to zero. This

TABLE I
Detection by condition and input (150 held-out frames).

Condition	Input	AP50	AP75	Prec.	Rec.
Day	Fused	99.8	85.8	0.993	0.989
Day	Thermal only	89.0	50.5	0.944	0.731
Night	Fused	94.3	58.3	0.916	0.869
Night	Thermal only	89.0	50.5	0.944	0.731
Washout	Fused	61.8	25.1	0.991	0.386
Washout	Thermal only	0.0	0.0	0.000	0.000

TABLE II
Effect of removing each component (paired mean ± 95% CI, 20 seeds).

Component removed: measure	Full	Ablated	Change
Failover (UAV lost): coverage	98.4%	87.4%	-11.0 ± 2.6
whole-arena recall	0.962	0.840	-0.122 ± 0.032
Hysteresis†: sector flips	2.5	13.5	$+11.0 \pm 3.5$
route changes	4.8	25.4	$+20.6 \pm 6.4$
mission time	362 s	392 s	$+30 \pm 25$ s
EW weighting (α=0): min. battery	18.8%	17.9%	-0.8 ± 0.7
DTN buffer (5 km): operator knows	98.0%	3.4%	-94.7 ± 1.2
swarm-wide consensus	100%	22.1%	-77.9 ± 1.3
Forwarding (5 km): alert latency	103 s	109 s	$+7 \pm 2$ s

† oracle planner, sensor noise $\sigma = 2\%$ (see text).

is modality-masked inference of one fused network, not separately trained models.

VI. Results

A. Perception

Table I shows that the fused model reaches 99.8 AP50 in daylight. Fusion beats thermal-only by about 11 points, so the visible plane adds information. Removing visible light costs only 5.5 points because thermal carries the detection; degraded thermal contrast is the hardest case (recall falls to 0.39 while precision stays 0.99). The RGB-only row is zero because the network was never trained without thermal; it says nothing about RGB's value, and it is why trained single-modality baselines remain to be done (Section VIII).

B. Reference mission

With no failure, the decentralized swarm covers 100% of the arena in 353±31 s (mean ± s.d. over 20 seeds). End-to-end precision, recall and F1 are all 0.980±0.015 (whole-arena recall counts unsearched victims), the mean localization error is 1.45±0.15 m (RMSE 1.86±0.32 m), the first victim is detected after 8.6±7.2 s, UAVs use 13.7±1.4% of their battery on average, and the operator learns of 100% of the reports. Localization error here measures the box-to-sector mapping, not geolocation, because each sector is one static frame.

C. What each component buys

Table II gives paired effects of removing a component.

Failover matters when a UAV is lost: coverage after the loss is 98.4% with failover and 87.4% without (Fig. 3); without it the lost UAV's sectors are never searched.

Hysteresis removes churn, not flight time. Under the oracle planner it cuts partition commits about tenfold (296 to 31 per mission at $\sigma = 2\%$) and sector flips from 13.5 to 2.5, with a mission-time change within noise. Under the decentralized planner claims are sticky, so there is little churn for it to remove (0.2 versus 0.4 flips), and the ablation shows almost nothing; this is a property of the planner, not evidence that hysteresis is useless.

EW weighting is a trade-off rather than a free gain. Sweeping α from 0 to 3 raises the minimum final battery from 15.6% to 20.3% and cuts low-battery aborts from 0.95 to 0.30 per mission, but lengthens the mission from 377 to 488 s because healthy UAVs carry more of the work; at $\alpha = 1$ the benefit is modest ($+0.8 \pm 0.7$ points of minimum battery).

D. Networking

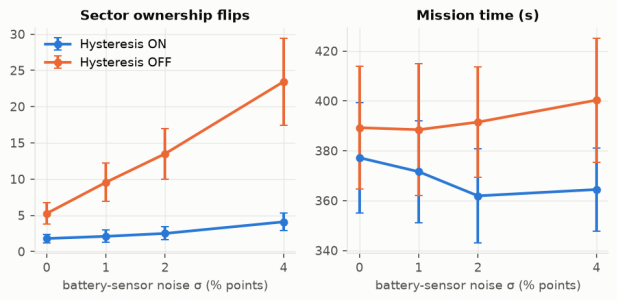
At these ranges the first-attempt delivery rate to the GCS is only 3–6%, so the swarm is mostly disconnected in flight (Fig. 4). Without a delay-tolerant buffer the operator learns of 3.4% of victims at 5 km and the swarm-wide consensus is 22%; with it, 98% and 100%. Buffering is by far the largest single effect we measured.

Store-carry-forward lowers alert latency (mean 41 versus 59 s at 3 km, 34 versus 41 s at 1.5 km, 103 versus 109 s at 5 km) and protects alerts when the finder is lost: with a UAV lost at 60 s the operator learns of 99.3% of reports at 3 km with forwarding and 97.3% without (98.9% versus 97.6% at 1.5 km). The older instant relay through a UAV that happens to sit between the endpoints adds nothing in a sparse swarm. Forwarding needs buffer: capped at 5, 15, 40 and 120 copies per UAV, the operator learns of 22.5%, 41.5%, 83.4% and 99.3% of reports (3 km, UAV lost), so the benefit is real but not cheap.

E. The cost of decentralization

Table III compares the decentralized planner with the oracle. With no failure they are equivalent (the decentralized swarm is slightly quicker because it does not re-plan in flight). After a loss the price of not knowing is time: the survivors cannot know UAV 2 is gone until it is overdue at base, so failover takes minutes and the mission is 133 s longer ($+133 \pm 52$ s) at 5 km. Coverage falls by 1.6 ± 2.9 points, and in a few seeds sectors stay unsearched because the survivors have no battery for a second sortie by the time they learn of the loss (for instance when the lost UAV carried the most charge); the oracle avoids this by re-planning at once. The gap narrows as the arena shrinks, because UAVs meet more often.

Lease slack. Slack trades detection speed against false alarms. At 20, 60, 120 and 240 s, the mean failover latency is 192, 231, 293 and 411 s, and the share of suspicions that were wrong falls from 11.2 of 15.6 to 0.4 of 4.7 per mission.



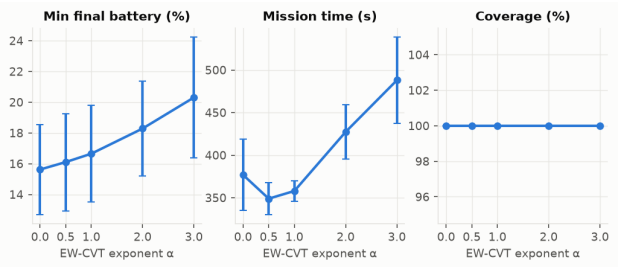


Fig. 2. Left: sector-ownership flips and mission time with hysteresis on and off as battery-sensor noise grows (oracle planner; error bars are 95% intervals). Right: effect of the EW-CVT exponent α on minimum final battery, mission time and coverage.

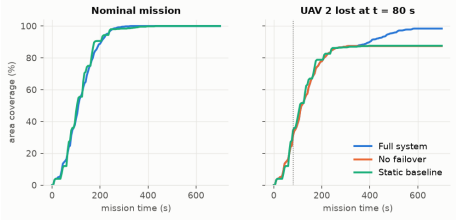


Fig. 3. Area coverage over time without a failure (left) and with UAV 2 lost at 80 s (right). Only variants with failover recover the lost area.

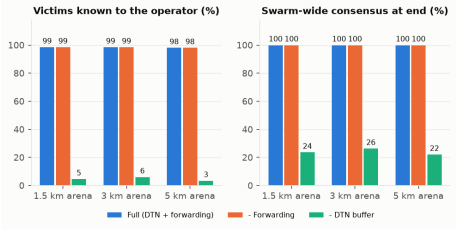


Fig. 4. Victims known to the operator (left) and swarm-wide consensus (right) by arena size. Without the delay-tolerant buffer almost nothing reaches the operator.

Most suspicions are of UAVs that were fine, since a UAV that landed looks like one that crashed to anyone still in flight. They cost little because released work is taken only at base: duplicate scans stay at about one per mission. Beacon interval hardly matters (mission time 532, 554, 587 and 629 s at 2, 5, 15 and 60 s) until it becomes comparable to the contact duration.

VII. Discussion and Limitations

Three results are robust in this evidence: buffering is essential in a sparsely connected swarm; losing a UAV costs real coverage unless others take its work; and not knowing a loss, rather than the loss itself, is what decentralization pays for. Two results are weaker than a design-level reading might suggest: hysteresis shows an effect only where partitions are recomputed from noisy readings every tick, and energy weighting trades speed for reserve rather than improving both.

Limits of the evidence. (1) The simulator is kinematic: constant speeds, linear battery drain, no wind, no obstacles beyond a probabilistic radio penalty. (2) Each sector

TABLE III
Decentralized (per-UAV views) versus oracle planner.

Scenario	Planner	Cover. %	Time (s)	Failover (s)
5 km, no loss	Oracle	100.0	367	–
	Per-UAV	100.0	353	–
5 km, loss	Oracle	100.0	421	instant
	Per-UAV	98.4	554	231
3 km, loss	Oracle	100.0	236	instant
	Per-UAV	99.0	326	143
1.5 km, loss	Oracle	100.0	146	instant
	Per-UAV	99.2	171	102

shows one static dataset frame, so mission-level precision and recall mostly reflect the detector on those frames plus which sectors were visited, and localization error measures a box-to-sector mapping. (3) The perception ablation masks one plane of a single fused network; train and validation frames come from the same site, so absolute numbers are optimistic. (4) Network results depend on the radio model. (5) The decentralized design simplifies in places: UAVs launch together and share the initial partition, beacons are single-hop, alerts are still addressed to every UAV whether alive or not, and forwarding reads the receiver's holdings as the summary exchanged on contact. The lease detector is one design among several. (6) The 15% reserve rule and the noise levels are scenario assumptions.

VIII. Ongoing and Future Work

- Trained single-modality baselines. Train RGB-only and thermal-only models alongside the fused one to replace masked inference with a proper modality ablation.
- Faster, better failure detection. Let beacons carry third-party state, use rendezvous points, and estimate contact intervals so that detection need not wait for an overdue return.
- Hardware-in-the-loop. Port the claim and lease protocol to the ArduPilot/ns-3 path derived from FlyNetSim [16] to test it under real flight dynamics and network stacks.
- Realistic imaging. Replace one static frame per sector with imagery tied to the UAV's position and a camera

model, so localization is geolocation.
- Richer energy and environment models (wind, payload, return-trip margin), and statistical checks of the scaling behaviour, whose small non-monotonicity between 4 and 6 UAVs is unexplained.

IX. Reproducibility

All numbers in this paper are produced by scripts in the project repository: a detector evaluation on 150 held-out frames, an ablation harness that runs 1,320 paired missions in a few minutes once detections are cached, and a report generator that writes the tables and figures used here. Every component is a command-line switch of the mission simulator (--no-hysteresis, --no-failover, --static, --no-dtn, --no-forwarding, --planner centralized, --alpha, --lease-slack, --beacon-interval, --buffer-cap), and the automated test suite covers the agent logic, forwarding and failover.

X. Conclusion

We built and evaluated a simulated search-and-rescue swarm with early-fusion RGB+thermal perception, a delay-tolerant mesh with store-carry-forward alert relaying, and a decentralized controller with no central planner. Every component has a switch, and over 1,320 paired missions the measured effects are mostly clear: buffering is essential, failover recovers coverage, forwarding speeds and protects alerts, and decentralization costs minutes of failover latency in exchange for removing a single point of failure. We report the unfavourable results and the limits of a simulation-only study, and we list the next steps needed to move toward flight.

References

[1] S. Hwang, J. Park, N. Kim, Y. Choi, and I. S. Kweon, "Multispectral pedestrian detection: Benchmark dataset and baseline," in Proc. IEEE CVPR, 2015.

[2] J. Liu, S. Zhang, S. Wang, and D. N. Metaxas, "Multispectral deep neural networks for pedestrian detection," in Proc. BMVC, 2016.

[3] L. Zhang et al., "Weakly aligned cross-modal learning for multispectral pedestrian detection," in Proc. IEEE ICCV, 2019.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, 2016.

[5] G. Jocher and J. Qiu, "Ultralytics YOLO11," software, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] S. Speth, A. Gonçalves, B. Rigault, S. Suzuki, M. Bouazizi, Y. Matsuo, and H. Prendinger, "Deep learning with RGB and thermal images onboard a drone for monitoring operations," J. Field Robot., vol. 39, no. 6, pp. 840–868, 2022; dataset: https://www.nii-cu-multispectral.org

[7] S. P. Lloyd, "Least squares quantization in PCM," IEEE Trans. Inf. Theory, vol. 28, no. 2, pp. 129–137, 1982.

[8] Q. Du, V. Faber, and M. Gunzburger, "Centroidal Voronoi tessellations: Applications and algorithms," SIAM Rev., vol. 41, no. 4, pp. 637–676, 1999.

[9] J. Cortés, S. Martínez, T. Karatas, and F. Bullo, "Coverage control for mobile sensing networks," IEEE Trans. Robot. Autom., vol. 20, no. 2, pp. 243–255, 2004.

[10] J. Cortés, S. Martínez, and F. Bullo, "Spatially-distributed coverage optimization and control with limited-range interactions," ESAIM Control Optim. Calc. Var., vol. 11, no. 4, pp. 691–719, 2005.

[11] F. Aurenhammer, "Power diagrams: Properties, algorithms and applications," SIAM J. Comput., vol. 16, no. 1, pp. 78–96, 1987.

[12] K. Fall, "A delay-tolerant network architecture for challenged internets," in Proc. ACM SIGCOMM, 2003.

[13] A. Vahdat and D. Becker, "Epidemic routing for partially connected ad hoc networks," Duke Univ., Tech. Rep. CS-2000-06, 2000.

[14] T. Spyropoulos, K. Psounis, and C. S. Raghavendra, "Spray and wait: An efficient routing scheme for intermittently connected mobile networks," in Proc. ACM SIGCOMM WDTN, 2005.

[15] K. Kosaka, Y. Noda, T. Yokotani, and K. Ishibashi, "Implementation and evaluation of the control mechanism among distributed MQTT brokers," IEEE Access, vol. 11, 2023, doi: 10.1109/ACCESS.2023.3335273.

[16] S. Baidya, Z. Shaikh, and M. Levorato, "FlyNetSim: An open source synchronized UAV network simulator based on ns-3 and Ardupilot," in Proc. ACM MSWiM, 2018, pp. 37–45.

[17] T. S. Rappaport, Wireless Communications: Principles and Practice, 2nd ed. Prentice Hall, 2002.

[18] M. Lyu, Y. Zhao, C. Huang, and H. Huang, "Unmanned aerial vehicles for search and rescue: A survey," Remote Sens., vol. 15, no. 13, art. 3266, 2023.

[19] M. A. Goodrich et al., "Supporting wilderness search and rescue using a camera-equipped mini UAV," J. Field Robot., vol. 25, no. 1–2, pp. 89–110, 2008.

[20] T. M. Cabreira, L. B. Brisolara, and P. R. Ferreira Jr., "Survey on coverage path planning with unmanned aerial vehicles," Drones, vol. 3, no. 1, art. 4, 2019.

[21] W. Burgard, M. Moors, C. Stachniss, and F. E. Schneider, "Coordinated multi-robot exploration," IEEE Trans. Robot., vol. 21, no. 3, pp. 376–386, 2005.

[22] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," Int. J. Robot. Res., vol. 23, no. 9, pp. 939–954, 2004.

[23] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," IEEE Trans. Robot., vol. 25, no. 4, pp. 912–926, 2009.

[24] R. Olfati-Saber, J. A. Fax, and R. M. Murray, "Consensus and cooperation in networked multi-agent systems," Proc. IEEE, vol. 95, no. 1, pp. 215–233, 2007.

[25] İ. Bekmezci, O. K. Sahingoz, and Ş. Temel, "Flying ad-hoc networks (FANETs): A survey," Ad Hoc Netw., vol. 11, no. 3, pp. 1254–1270, 2013.

[26] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: Opportunities and challenges," IEEE Commun. Mag., vol. 54, no. 5, pp. 36–42, 2016.

[27] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," IEEE Wireless Commun. Lett., vol. 3, no. 6, pp. 569–572, 2014.

[28] X. Jia, C. Zhu, M. Li, W. Tang, and W. Zhou, "LLVIP: A visible-infrared paired dataset for low-light vision," in Proc. IEEE/CVF ICCV Workshops, 2021.

[29] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in Proc. IEEE/CVF CVPR, 2023, pp. 7464–7475.

[30] M. Schwager, D. Rus, and J.-J. Slotine, "Decentralized, adaptive coverage control for networked robots," Int. J. Robot. Res., vol. 28, no. 3, pp. 357–375, 2009.

[31] A. Lindgren, A. Doria, E. Davies, and S. Grasic, "Probabilistic routing protocol for intermittently connected networks," IETF RFC 6693, 2012.

[32] A. Demers et al., "Epidemic algorithms for replicated database maintenance," in Proc. ACM PODC, 1987, pp. 1–12.

[33] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," J. ACM, vol. 43, no. 2, pp. 225–267, 1996.

[34] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ accrual failure detector," in Proc. IEEE SRDS, 2004, pp. 66–78.

[35] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in Proc. ACM SOSP, 1989, pp. 202–210.